Part 2

APRENDE PYTHON

Estructuras de datos



APRENDE PYTHON

Estructuras de datos

Ya sabes guardar **un** dato. Ahora aprenderás a guardar **muchos** y bien organizados. Para eso existen las **estructuras de datos**: cajas especiales, cada una para un trabajo distinto.

`[ ]` **Lista:** valores en orden, por posición.

`{ }` **Diccionario:** pares nombre: valor.

`( )` **Tupla:** datos que no cambian.

`{ }` **Set:** valores únicos, sin repetir.

💡 Recuerda nuestro dataset: `novelas` es una **lista** de **diccionarios**. ¡Es una estructura dentro de otra!

Listas | Diccionarios | Tuplas | Sets

Milagros Espinoza Cunya

APRENDE PYTHON

Estructuras de datos

1 Listas

¿Qué es una lista?

Una colección **ordenada** de valores, entre corchetes **[ ]**. Cada valor tiene una posición llamada **índice**.

Problema

Queremos guardar todos los títulos juntos y sacar uno por su posición.

La Solución:

Creamos la lista con **[ ]** y accedemos a un valor con su **índice** entre corchetes.

```
titulos = ["Rubí", "Teresa", "Rebelde"]
```

```
la lista        posición 0 = la primera
   ↓                    ↓
titulos              [0]
```

Resultado:

```
>>> titulos[0]
'Rubí'
>>> titulos[-1]
'Rebelde'   # -1 = el último
```

DATAZO

El índice empieza en **0**, no en 1. Por eso **titulos[0]** es el primero, y **-1** siempre es el último. Esto es lo que más confunde al inicio.

Milagros Espinoza Cunya

APRENDE PYTHON

Estructuras de datos

2 Listas: métodos

¿Qué es un método?

Una **acción** que la lista ya sabe hacer. Se llama con un punto: **lista.metodo()**.

Problema

Queremos agregar, cambiar y borrar títulos de la lista.

La Solución:

Usamos métodos como **append()** y el índice para modificar o borrar.

```
titulos.append("Betty")       # agrega al final
titulos[1] = "La Usurpadora"  # modifica
titulos.remove("Rubí")        # elimina
```

`.append()` agrega al final | `.remove()` borra por valor | `len()` cuántos hay

Resultado:

```
>>> len(titulos)
3   # quedó con 3 títulos
```

DATAZO

append() siempre agrega al final. Si quieres meterlo en una posición exacta, usa **insert(posición, valor)**.

Milagros Espinoza Cunya

APRENDE PYTHON

Estructuras de datos

3 Diccionarios

¿Qué es un diccionario?

Guarda pares **clave: valor**. Accedes por el **nombre** (la clave), no por un número de posición.

Problema

Queremos los datos de **una** novela y poder agregar o cambiar un dato suyo.

La Solución:

Usamos la **clave** entre corchetes para leer, agregar o modificar.

```
el diccionario   la clave que buscas
      ↓                  ↓
   novela          ["titulo"]
```

```
novela["anio"] = 2004    # agrega clave nueva
novela["rating"] = 9.9   # modifica la clave
```

Resultado:

```
>>> novela["titulo"]
'Rebelde'
```

DATAZO

La gran diferencia: la **lista** accede por POSICIÓN (número); el **diccionario**, por NOMBRE (clave).

Milagros Espinoza Cunya

APRENDE PYTHON

Estructuras de datos

4 Tuplas y Sets

Dos estructuras más

La **tupla ( )** es una lista que **NO se puede cambiar**. El **set { }** no permite **valores repetidos**.

Tupla — datos fijos

```
coords = (10, 20)   # no se puede modificar
```

Set — sin duplicados

```
generos = {"drama", "drama", "acción"}
# resultado -> {"drama", "acción"}
```

Un `set` usa llaves { } pero `sin` dos puntos; el diccionario sí lleva `clave: valor`.

DATAZO

Usa **tupla** para datos que no deben cambiar (una fecha, coordenadas). Usa **set** cuando necesites quedarte solo con valores únicos.

Milagros Espinoza Cunya

APRENDE PYTHON

Estructuras de datos

5 Recorrer la data

El patrón más importante

Una **lista de diccionarios** es como una tabla: cada diccionario es una **fila**. Con un **for** la recorres toda.

Problema

Queremos mostrar el título y el rating de todas las novelas juntas.

La Solución:

Recorremos la lista con **for** y de cada novela leemos sus claves.

```
for novela in novelas:
    print(novela["titulo"], "-", novela["rating"])
```

Resultado:

```
Rubí - 8.9
Teresa - 9.0
Rebelde - 9.8
```

DATAZO

Este patrón (**for** + diccionario) es el que más usarás en análisis de datos: recorrer filas y leer sus columnas.

Milagros Espinoza Cunya

APRENDE PYTHON

Estructuras de datos

Las 4 estructuras que debes dominar:

Estructura	Símbolo	¿Cuándo usarla?
Lista	`[ ]`	Varios valores en orden; acceso por posición.
Diccionario	`{ }`	Pares clave: valor; acceso por nombre.
Tupla	`( )`	Datos fijos que no cambian (inmutable).
Set	`{ }`	Valores únicos, sin duplicados.

¡Guarda este post, dale like, compártelo y sígueme para la Parte 3!

Milagros Espinoza Cunya